



GOLF *API PROJECT*

JACK FEINBERG + BENJAMIN GOLDENSTEIN + ETHAN ZUCKERMAN

OUR INITIAL GOALS

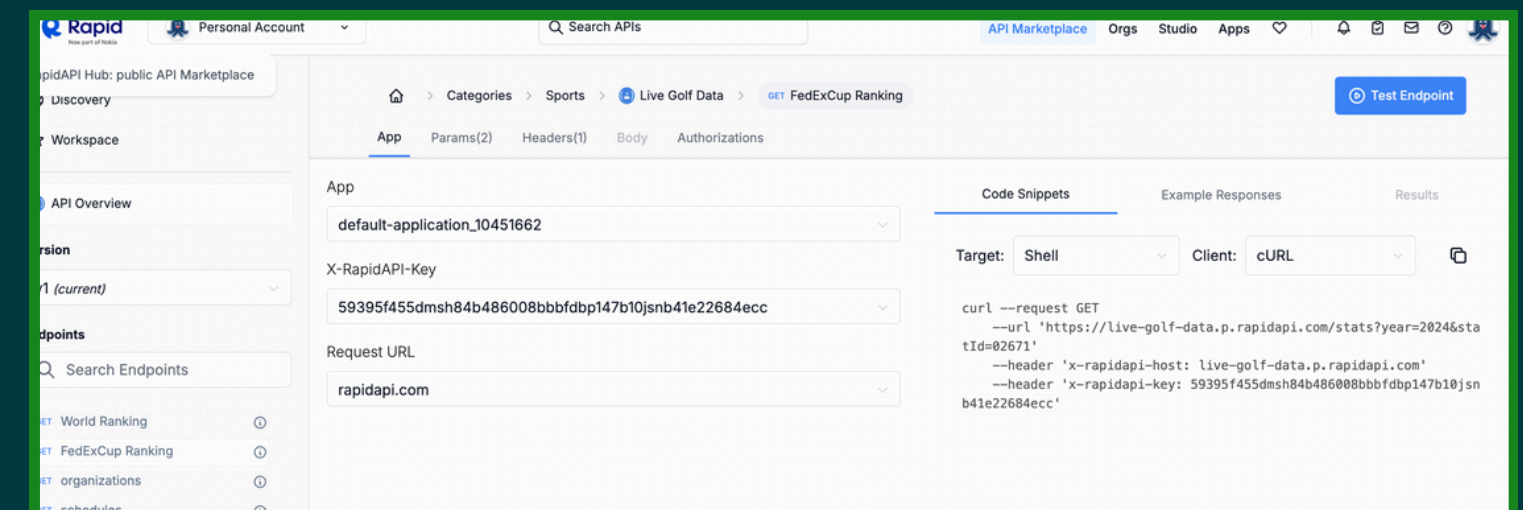
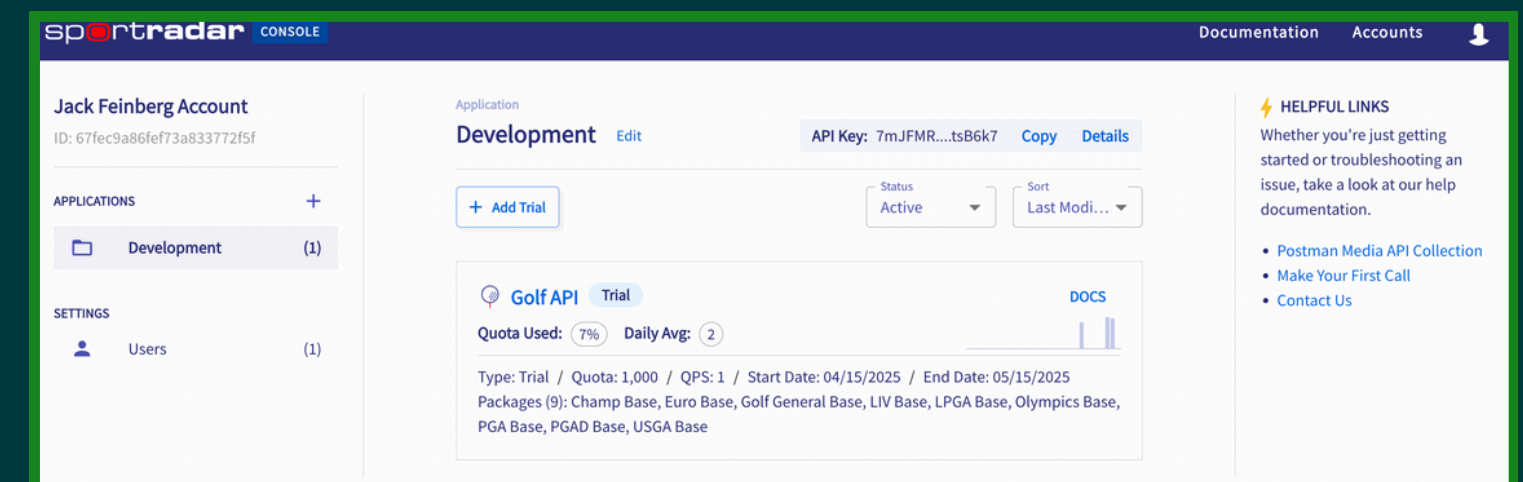
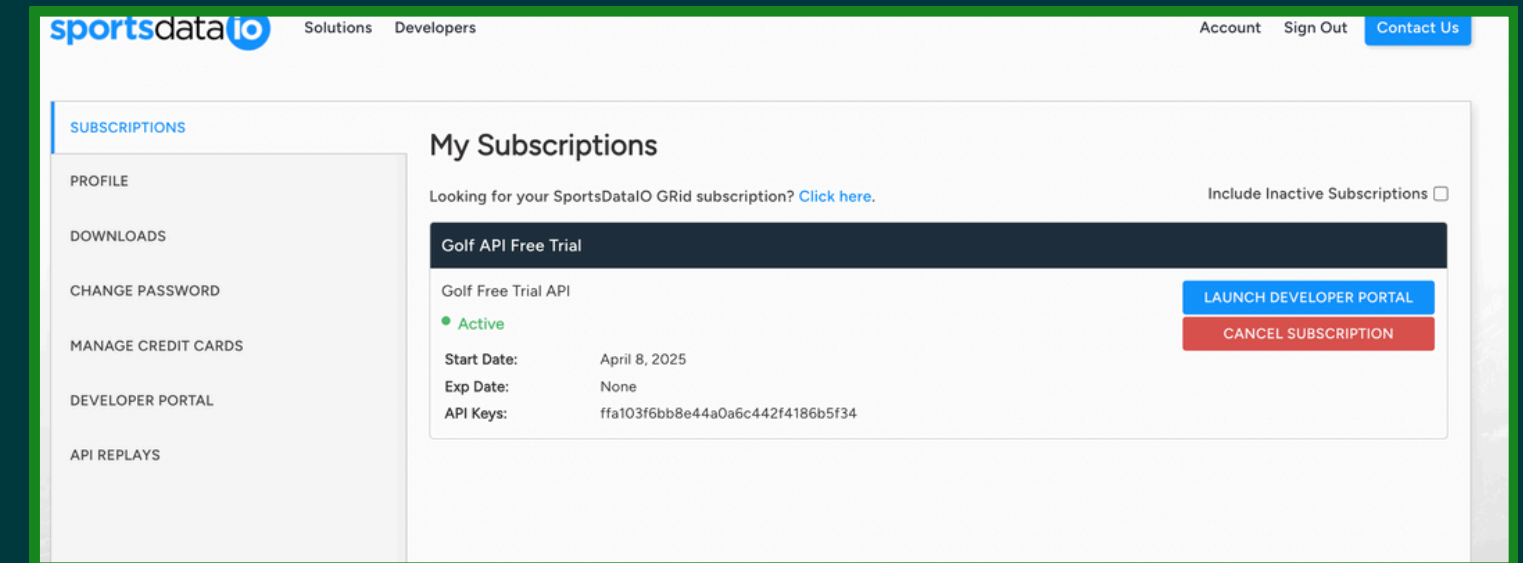
APIs and Websites + Data Gathering Goals...

Our original plan was to use three different APIs for baseball-related data. First, we planned to scrape historical team performance data from Baseball Reference using BeautifulSoup. Second, we intended to use the MLB-StatsAPI to collect how teams performed in the first two weeks of the current season. Third, we planned to use an MLB Data API to gather projected pitching and hitting statistics for individual players. The goal was to combine all of this data to calculate and visualize which MLB teams had the highest probability of winning their division, league, or the overall championship. Our calculations were going to use April performance trends and player projections to make these predictions.

AFTER CONSIDERATION...

APIs and Websites + Data Gathering Goals...

Instead of working with baseball, we shifted our project to focus on professional golf. We found these APIs to be easier to use and we thought this data could be compiled together in a cleaner manner. We used three different APIs: SportsDataIO to pull finalized player tournament scores, the Sportradar API to get official world golf rankings, and the Live Golf Data API to get FedEx Cup rankings and player statistics like wins and top 10s. Our new goal became identifying the top players based on their performance in the current season and predicting who is most likely to win future events. We organized all of this data into a single SQLite database, calculated metrics such as average points per event, and built visualizations comparing players' finishes and projected likelihood of future success.



API USAGE + GOALS ACHIEVED

We had success with all three APIs when it came to gaining access and retrieving reliable data. Each API had enough information to pull at least 100 records, which allowed us to build out our individual database tables. Originally, we had separate databases for each API, but as the project progressed, we realized it made more sense to centralize everything into one database with separate tables to make analysis and joins more efficient.

SPORTS.IO API

We used this API to collect individual player scores for multiple tournaments. Specifically, we gathered player names, their unique IDs, the tournament ID, and total score for each tournament round. We then wrote these into a table within our unified database and used the data to identify top 10 finishes and wins by comparing scores across tournaments.

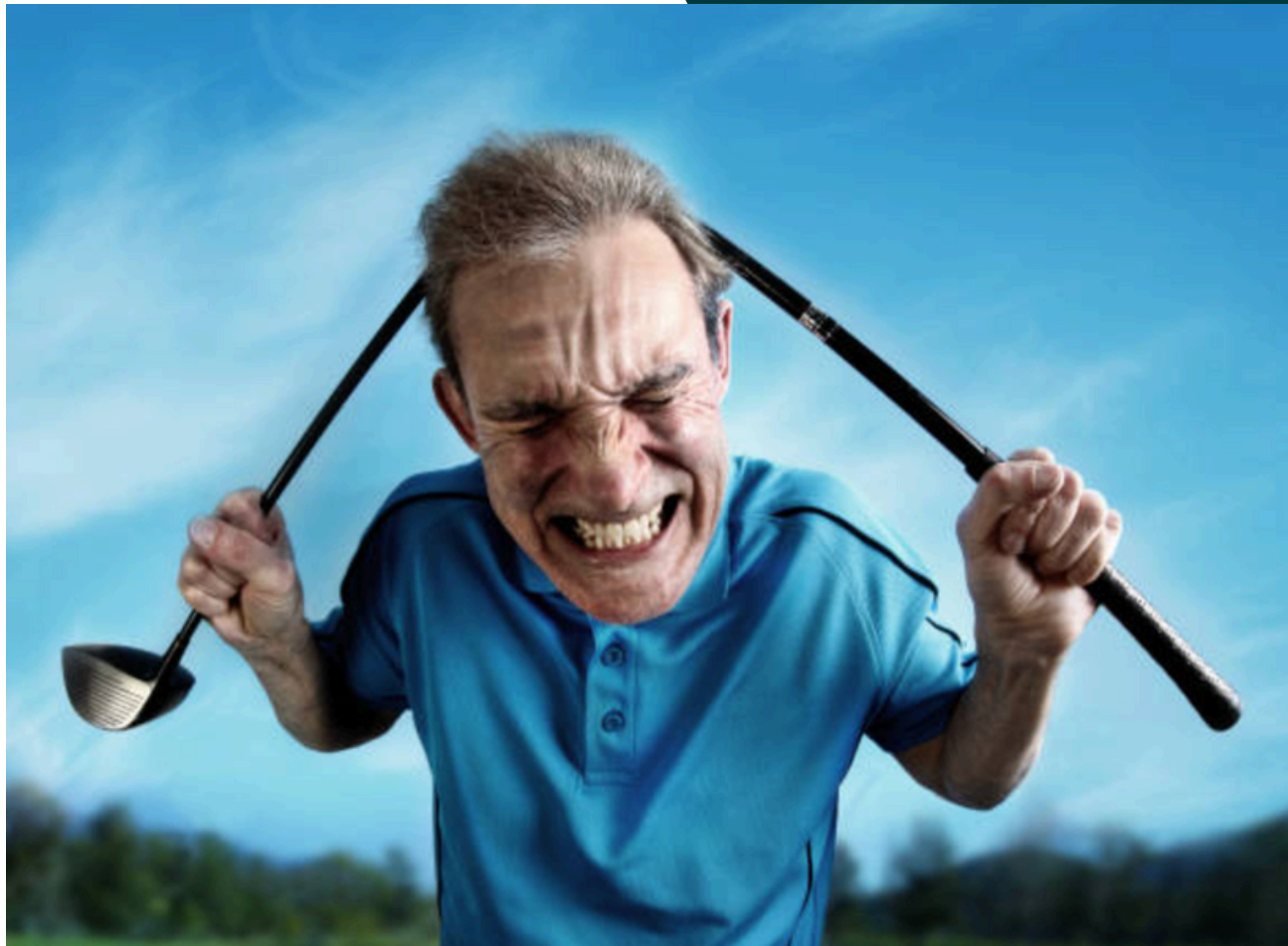
SPORT RADAR API

From Sport Radar, we pulled global ranking data for each player, including first and last name, a locally generated ID, events played, and total ranking points. We stored this in a table and used it to compute average points per event for each player. This became part of our prediction model for projecting player performance over the rest of the season.

LIVE GOLF DATA API

We used this API to collect FedEx Cup standings. The data included player names, their current and previous ranks, number of wins, and top 10s. We used this info to build visualizations and calculate FedEx Cup “achievement scores” by summing wins and top 10s. This helped give us a snapshot of player momentum across the season.

PROBLEMS FACED



1.

Not Enough Data from a Single API Call

One of the APIs has less than 100 lines of code in the same tournaments. To reach the 100 required lines of code for the project, we had to figure out how to move from one tournament to the next to make sure that all of the necessary data was being pulled.

2.

Inconsistent Player Coverage Across APIs

One challenge was that the Live Golf Data API listed players not found in the World Rankings, so we had to skip unmatched names when writing to the database. Another issue came from the Sports.io API, which didn't return enough players in one tournament, so we added logic to move to the next tournament once 100 lines were collected.

3.

Inconsistent Name Formatting Across APIs

One challenge we faced was that different APIs used slightly different name formats (like "first_name" vs "firstName"), so we had to standardize all names when joining tables to avoid mismatches.

CALCULATIONS & CSV CREATION

Sports 10	Calculated wins and top 10 finishes by comparing total scores across multiple tournaments — players with the lowest score were counted as winners, and the top 10 scores were flagged each time.
Sport Radar	Used total points and events played to calculate each player’s average points per event, which we later used in projections.
Live Golf Data	Used the number of wins and top 10s already provided to generate a FedEx “achievement score” by summing those two values

The goal of these calculations was to create clean, analyzable datasets from our raw API pulls. We wrote each result into its own CSV file —
golfer_performance.csv (wins and top 10s from Sports.io),
player_averages.csv (average points from SportRadar), and
fedex_total_achievement.csv (achievement score from Live Golf Data).

From there, we used these CSVs to make our prediction csv and the visualizations.

CALCULATIONS & CSV CREATION

```
# Write to CSV
✓ with open(output_path, mode="w", newline="") as csvfile:
    writer = csv.writer(csvfile)
    writer.writerow(["Player", "Events Played", "Total Points", "Average Points"])

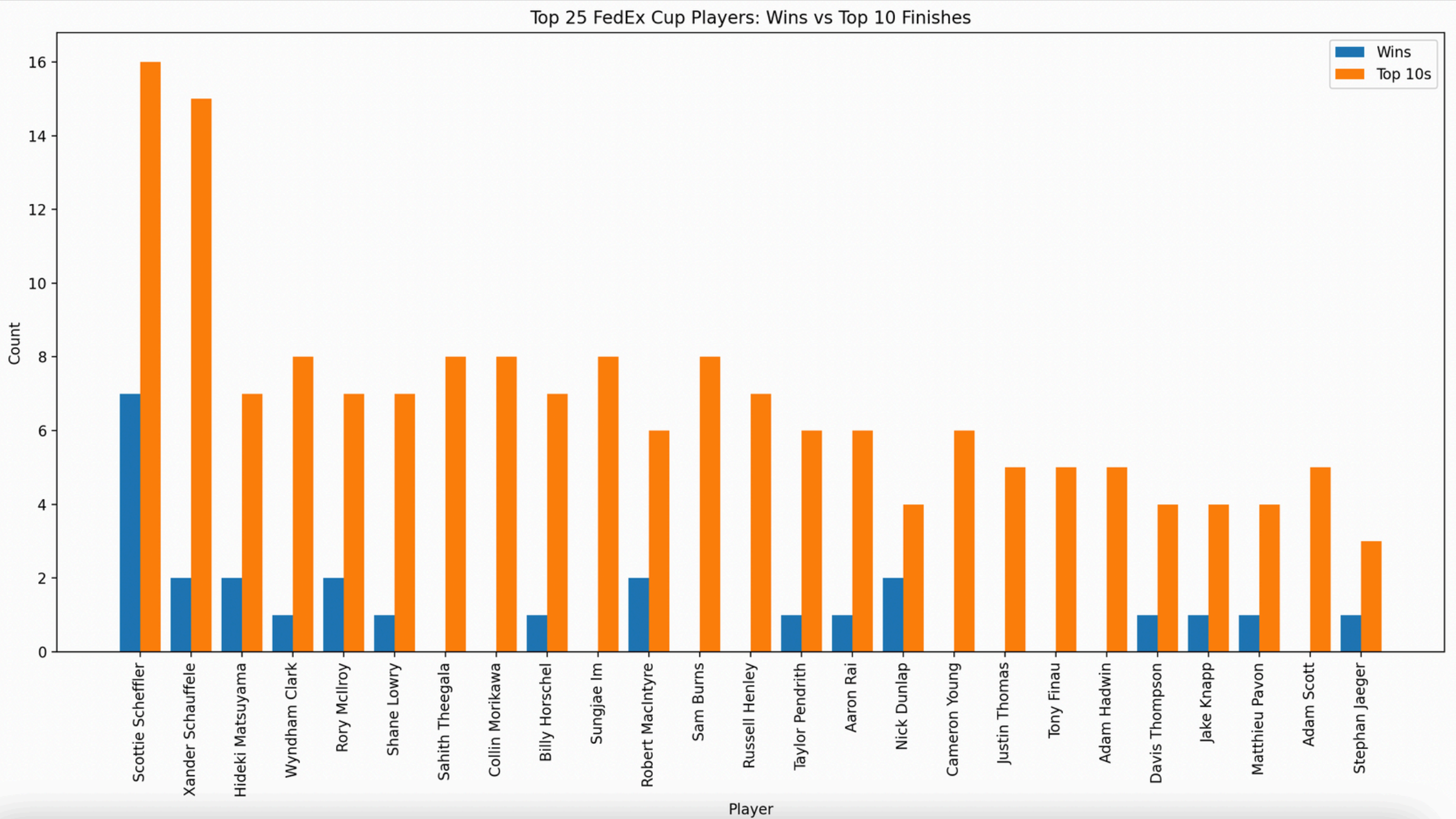
    for row in rows:
        first_name, last_name, events, points = row
        full_name = f"{first_name} {last_name}"
        try:
            avg_points = round(points / events, 2) if events > 0 else 0.0
        except Exception:
            avg_points = 0.0
        writer.writerow([full_name, events, round(points, 2), avg_points])
```

```
# Compute average points and likelihood score
df["AveragePoints"] = df.apply(lambda row: round(row["TotalPoints"] / row["EventsPlayed"], 2) if row["EventsPlayed"] > 0 else 0.0, axis=1)

df["LikelihoodScore"] = (
    df["Wins"] * 10 +
    df["Top10s"] * 5 +
    df["AveragePoints"] * 3
)
```

```
# Calculate total achievements
df["TotalAchievements"] = df["Wins"] + df["Top10s"]
```

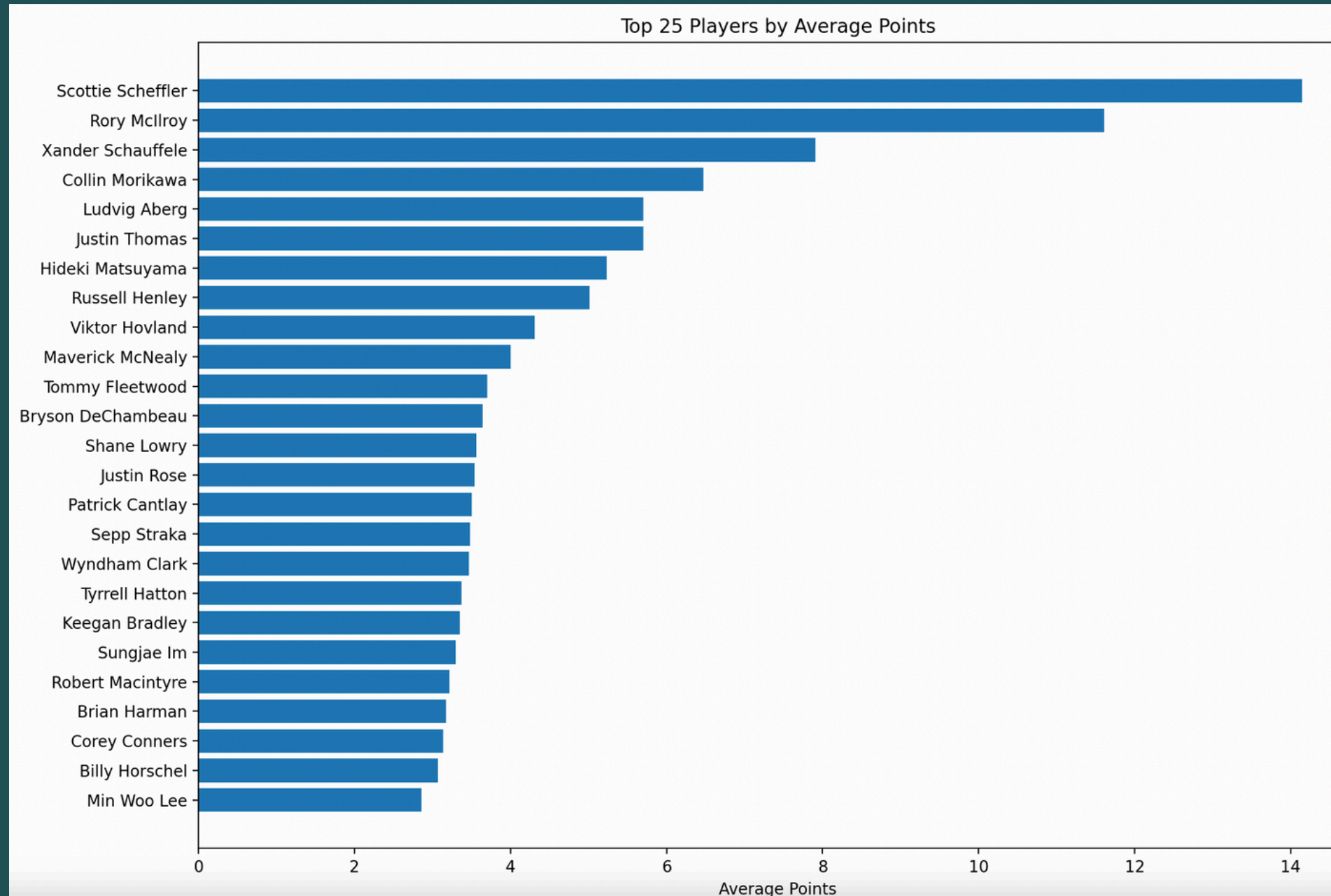

VISUALIZATIONS



FEDEX CUP DATA

THIS VISUALIZATION PULLS FROM FEDEX_TOTAL_ACHIEVEMENTS.CSV, WHICH WAS GENERATED USING DATA FROM THE LIVE GOLF DATA API. IT SHOWS THE TOP 25 FEDEX CUP PLAYERS BASED ON TOTAL ACHIEVEMENTS (WINS + TOP 10S), WITH EACH BAR PAIR COMPARING HOW MANY WINS AND TOP 10 FINISHES EACH PLAYER HAS THIS SEASON.

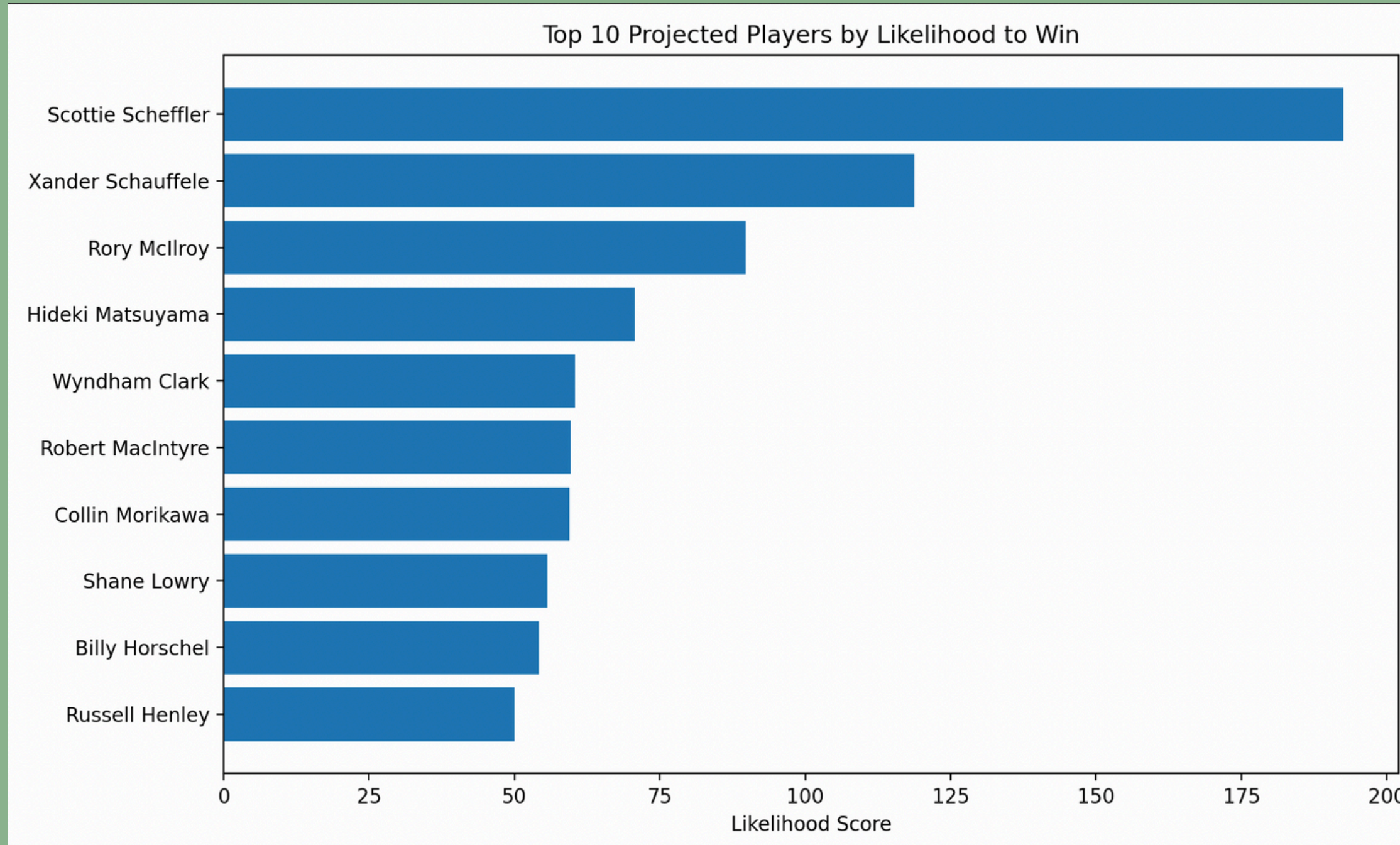
VISUALIZATIONS



TOP 25 CURRENT PLAYERS

THIS VISUALIZATION PULLS FROM `PLAYER_AVERAGES.CSV`, WHICH WAS GENERATED USING DATA FROM THE SPORTRADAR API. IT SHOWS THE TOP 25 PLAYERS RANKED BY THEIR AVERAGE POINTS PER EVENT – A METRIC WE CALCULATED BY DIVIDING TOTAL POINTS BY EVENTS PLAYED – GIVING A CLEARER VIEW OF WHO PERFORMS MOST EFFICIENTLY ACROSS TOURNAMENTS.

VISUALIZATIONS



REST OF SEASON PREDICTIONS

THIS VISUALIZATION PULLS FROM `GOLFER_WIN_PREDICTIONS.CSV`, WHICH COMBINES DATA FROM MULTIPLE APIS. IT USES THE "LIKELIHOOD SCORE" WE CALCULATED BY COMBINING WINS AND TOP 10S (FROM THE SPORTS.IO AND LIVE GOLF DATA APIS) WITH AVERAGE POINTS PER EVENT (FROM THE SPORTRADAR API). THE CHART SHOWS THE TOP 10 PLAYERS PROJECTED TO PERFORM BEST FOR THE REMAINDER OF THE SEASON, RANKED BY THEIR OVERALL SCORE, WHICH BALANCES CONSISTENCY, PAST PERFORMANCE, AND EFFICIENCY.

RUNNING THE CODE – WHAT IT DOES

API FOLDER

1	1Current_rankings.py	Pulls 25 current world ranking data from the Sportradar API and writes it to the All_Golf_Data.db under the players table.
2	2Live_GolfData.py	FedEx Cup standings data (wins, top10s, etc.) from the Live Golf Data API and writes 25 new records per run to the FedEx_Cup_Standings table in All_Golf_Data.db.
3	3Sports_io.py	Pulls tournament scores from the Sports.io API, adds 25 results at a time, and stores them in the scores table in All_Golf_Data.db, linked to the correct GolferID.

RUNNING THE CODE – HOW TO DO IT

API FOLDER

1	1Current_rankings.py	This is the first function in our code. Run this multiple times until there is over 100 lines of data produced in the DB.
2	2Live_GolfData.py	Then move on to the second file. Repeat the steps from the first file in order to get to that same target of having 100 lines of data. If you run it too quickly it might give an error but just give it a second and run again.
3	3Sports_io.py	Repeat the process from the first two files to get the necessary amount of data in the DB.

RUNNING THE CODE DATA PROCESSING FOLDER

1	1last_season_wins_plus_top10s	Processes FedEx_Cup_Standings data from the DB and writes fedex_total_achievements.csv.
2	2Calculate_average_world_points.py	Reads from the players table in All_Golf_Data.db and writes player_averages.csv, calculating each player's average points per event.
3	3Process_previous_tournaments.py	Reads the scores table for tournament IDs 628, 656, and 657, calculates wins and top 10s for each golfer, and outputs golfer_performance.csv.
4	4Predict_SeasonRankings.py	Merges golfer_performance.csv and player_averages.csv, computes a custom LikelihoodScore, and exports golfer_win_predictions.csv.

RUNNING THE CODE DATA PROCESSING FOLDER

1	1last_season_wins_plus_to p10s	This is much more simple. Just run this file and you should see a CSV file created in the folder.
2	2Calculate_average_world _points.py	Repeat that process again.
3	3Process_previous_tourna ments.py	Repeat that process again.
4	4Predict_SeasonRankings. py	This will also create a csv file using the data collected in the other ones to conduct a formula and calculate a prediction for how the players in the DB will perform the rest of the season.

RUNNING THE CODE WHAT IT DOES

VISUALIZATION FOLDER

1	1_2024_FedEx_Cup_Data.py	Visualizes the top 25 FedEx players by wins and top 10s using fedex_total_achievements.csv.
2	2_Top_25_current_players.py	Creates a bar chart showing the top 25 players by average points using player_averages.csv.
3	3_Predictions.py	Displays a horizontal bar chart ranking the top 10 golfers by LikelihoodScore using golfer_win_predictions.csv.

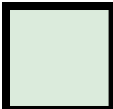
RUNNING THE CODE – HOW TO USE IT

VISUALIZATION FOLDER

1	1_2024_FedEx_Cup_Data.py	For the visualizations, just go to this file and click run and a attachment of the created data table should automatically pop up on your computer.
2	2_Top_25_current_players.py	Do the same for this one and it will display the second chart.
3	3_Predictions.py	This will display what was found in the prediction csv before. Visualizing which players have the best chances the rest of the season.

FUNCTION DOCUMENTATION

<u>Function</u>	<u>Input</u>	<u>Output</u>
get_last_player_id()	None	Integer (last player ID stored in DB)
fetch_and_store_data()	year, statId, start_index, batch_size, api_key, table_name	None (writes directly to SQLite DB)
fetch_scores()	None	List of score dictionaries from the API
setup_db()	None	SQLite connection object
insert_scores(conn, scores)	conn: database connection object scores: list of player score dictionaries	None (writes to DB)

 1Current_rankings.py

 2Live_GolfData.py

 3Sports_io.py

RESOURCES USED

Date	Issue Description	Location of Resource	Result (Did it solve the issue?)
4/10	Needed clarity understanding the process of scrapping the API and going about how to do it with our own API	https://www.youtube.com/watch?v=DqtIR0y0suo	Gave us a better understanding of how to go about the process for us to start working on it ourselves
4/15	Needed clarity in understanding how to go about the next step which was visualizing the data and making the graphs	http://geeksforgeeks.org/visualize-data-from-csv-file-in-python/	Read through these webpages to understand how to go about visualizing the data from our CSV files and it put us on the right track to getting it done
4/15	Kept trying to run the visualization code but getting errors	AI tool	Did not fully solve issue but helped us figure out how to solve it ourselves
4/16	Could not figure out how to move from one tournament to the next in the third API	AI Tool	For this case, we just inputted the error message into chat GPT and it was able to help us after a couple iterations of fixing the prompt.

GIT HUB REPO:

**[HTTPS://GITHUB.COM/JFEINY/GOLF_FINAL_PROJECT](https://github.com/JFEINY/GOLF_FINAL_PROJECT)
.GIT**

THANK YOU